API Interview Questions

1 What is an API?

- API stands for Application Programming Interface.
- It is a set of rules that allows different software applications to communicate with each other.
- It acts as a bridge between two systems.
- APIs save development time.
- APIs define how requests and responses should be structured, including methods, data formats, and authentication rules.

Example: A payment app using a bank API to transfer money.

2 What is REST API?

- REST stands for Representational State Transfer.
- REST APIs use HTTP methods for communication such as GET, POST, PUT, DELETE.
- REST APIs are stateless, meaning the server does not remember previous client requests.
- REST APIs usually return data in JSON format as it is lightweight and easy to read.

Example: GET /users/10 → Fetch user with ID 10.

3 What is the difference between API and REST API?

- API is a general concept, while REST API is a specific type of API.
- **An API can use different protocols** like HTTP, SOAP, GraphQL, WebSocket.
- **REST API follows REST architectural principles.**
- **All REST APIs are APIs, but not all APIs are REST APIs.**

Example: SOAP API is an API but not a REST API.

4 What are HTTP methods in API?

- GET → Used to retrieve data from the server.
- POST → Used to send new data to the server.
- PUT → Used to update an entire existing resource.
- PATCH → Used to partially update data.
- DELETE → Used to remove data.
- OPTIONS → Used to check allowed methods.
- HEAD → Similar to GET but returns headers only.

Example: DELETE /users/5

5 What is the difference between GET and POST?

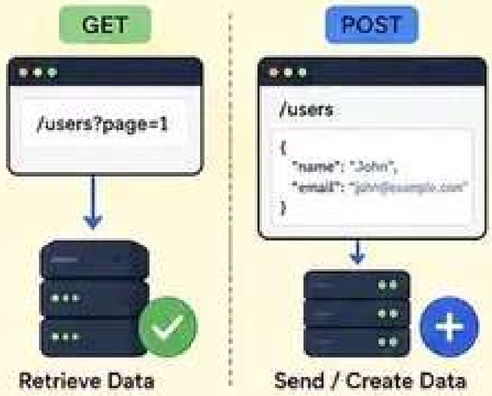
GET:

- Used to retrieve data from server.
- Data is sent in URL as query parameters.
- Less secure for sensitive data.
- Can be cached.
- Limited data size.

POST:

- Used to send or create data on server.
- Data is sent in request body.
- More secure than GET.
- Not cached by default.
- Supports larger data size.

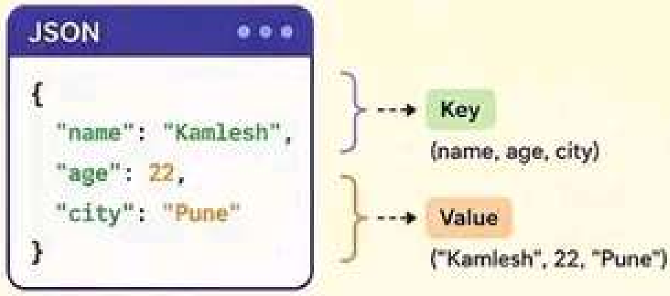
Example: GET /users → Fetch user list
POST /users → Create a new user



6 What is JSON?

- JSON stands for JavaScript Object Notation.
- It is a lightweight format used for data exchange.
- It is easy for humans to read and machines to parse.
- REST APIs commonly use JSON for request and response data.
- JSON stores data as key-value pairs.

Example: { "name": "Kamlesh", "age": 22, "city": "Pune" }



7 What is an endpoint in API?

- An endpoint is a specific URL where an API can be accessed.
- It represents a resource or operation.
- Clients send requests to endpoints.
- Different endpoints perform different tasks.

Example: https://api.example.com/users
Here, /users is the endpoint.

8 What is statelessness in REST API?

- Stateless means the server does not store client session information.
- Every request must contain all necessary information.
- Each request is independent of others.
- This improves scalability and performance.
- Server becomes easier to maintain.

Example: If authentication token is needed, client sends it with every request.

9 What is HTTP status code?

- HTTP status codes are server responses indicating the result of a request.
- They help clients understand whether the request was successful, redirected, has an error, or failed.
- Status codes are grouped into 5 categories (1xx to 5xx).
- They are an important part of API communication.

- 200 OK → Request successful.
- 201 Created → Resource created successfully.
- 400 Bad Request → Client sent invalid request.
- 401 Unauthorized → Authentication required.
- 403 Forbidden → Access denied.
- 404 Not Found → Resource not found.
- 500 Internal Server Error → Server-side issue.

10 What is the difference between 401 and 403?

401 Unauthorized

- User is not authenticated.
- Login or token is required.
- Authentication failed or missing.
- The client must authenticate first.

Example: No JWT token provided.

403 Forbidden

- User is authenticated but lacks permission.
- Server understood the request but denies access.
- User does not have rights to access the resource.

Example: Normal user trying to access admin panel.

11 What is authentication in API?

- Authentication is the process of verifying the identity of the user or system.
- It ensures that the request is coming from a valid user.
- Common methods include API Key, Basic Auth, OAuth, and JWT.
- It is the first step in API security.

Example: Sending JWT token in Authorization header.

12 What is authorization in API?

- Authorization is the process of determining what an authenticated user is allowed to do.
- It checks the permissions or role of the user.
- Even after authentication, user may not be authorized to access certain resources.
- It prevents unauthorized access to sensitive data.

Example: Only admin can delete a user.

13 What is the difference between PUT and PATCH?

PUT

- Used to update an entire resource.
- Client must send all fields, even unchanged ones.
- It replaces the existing resource completely.
- Idempotent: Multiple identical requests have the same effect.

Example: PUT /users/1
{ "name": "John", "age": 25 }
(Updates all fields)

PATCH

- Used to update partial data.
- Client sends only the fields that need to change.
- It modifies the existing resource partially.
- Not always idempotent.

Example: PATCH /users/1
{ "age": 25 }
(Updates only age field)

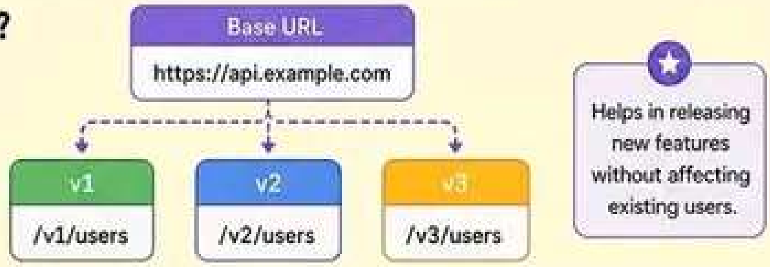
14 What is CRUD in API?

CRUD stands for Create, Read, Update, Delete.

- C **Create** → Add new data to the server (POST).
- R **Read** → Retrieve existing data (GET).
- U **Update** → Modify existing data (PUT/PATCH).
- D **Delete** → Remove data from the server (DELETE).

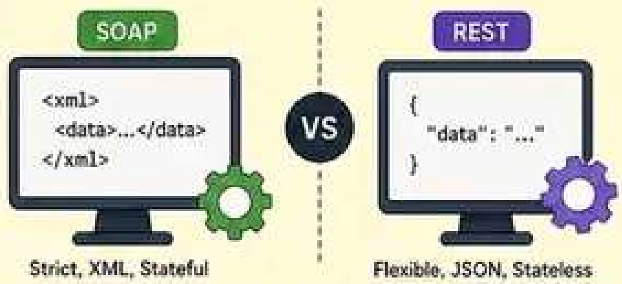
15 What is API versioning and why is it important?

- API versioning is a way to manage changes in API without breaking existing clients.
- It allows multiple versions of an API to coexist.
- It is important to maintain backward compatibility.
- Common approaches: URL versioning, Header versioning, Query parameter versioning.



16 What is the difference between SOAP and REST API?

SOAP API	REST API
SOAP stands for Simple Object Access Protocol.	REST stands for Representational State Transfer.
Uses XML as data format.	Uses JSON (or XML) as data format.
Uses strict protocols (HTTP, SMTP, etc.).	Uses HTTP protocol.
Stateful.	Stateless.
More secure (WS-Security).	Less secure by default (needs HTTPS, OAuth, etc.).
Complex and heavy.	Simple, lightweight, and easy to use.
Less performance.	Better performance and scalability.



Save this post to revise later!

17 What is JWT?

- JWT stands for JSON Web Token.
- It is a compact and self-contained way to securely transmit information between parties.
- It is commonly used for authentication and information exchange.
- JWT contains a header, payload, and signature.
- It is stateless and can be verified without storing session on the server.

Example: After login, server sends a JWT token. Client sends this token in every request.

18 What is OAuth 2.0?

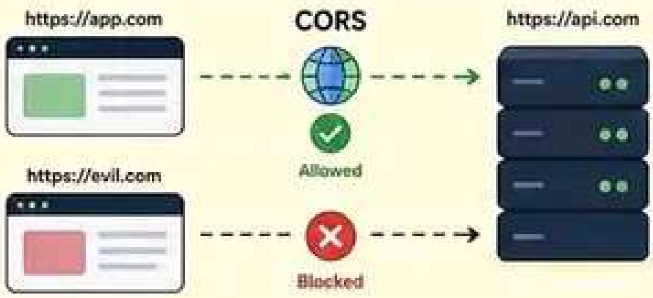
- OAuth 2.0 is an authorization framework, not an authentication protocol.
- It allows users to grant limited access to their resources without sharing their credentials.
- It is commonly used for third-party login (Google, Facebook, GitHub).
- It works using access tokens and refresh tokens.

Example: "Login with Google" uses OAuth 2.0.

19 What is CORS?

- CORS stands for Cross-Origin Resource Sharing.
- It is a security feature implemented by browsers.
- It allows or restricts web pages to make requests to a different domain.
- It is controlled by HTTP headers set by the server.

Example: Access-Control-Allow-Origin: https://example.com



20 What is rate limiting?

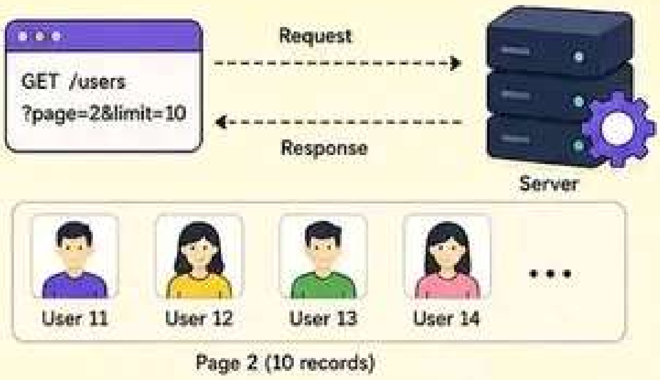
- Rate limiting is a technique to control the number of requests a client can make in a given time.
- It prevents abuse and protects the API from overuse.
- It improves performance and ensures fairness.
- It can be implemented by IP, user, or API key.

Example: 100 requests per minute per user.

21 What is pagination in API?

- Pagination is a technique used to divide large data into smaller chunks or pages.
- It improves performance and reduces server load.
- Common parameters: page, limit, offset.
- It helps clients load data step by step.

Example: GET /users?page=2&limit=10
→ Returns 10 users from page 2.



22 What is idempotency in API?

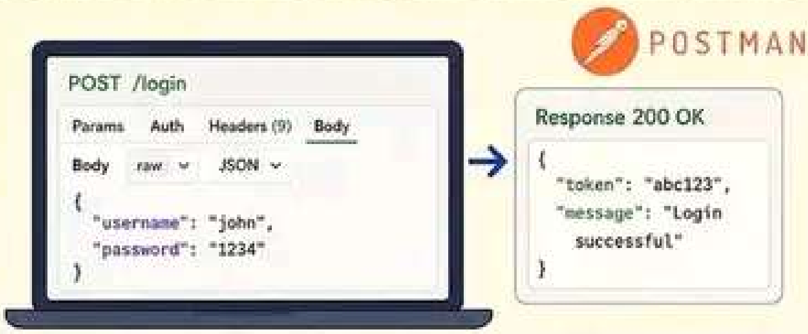
- An idempotent request produces the same result, even if sent multiple times.
- It ensures that repeated requests do not create duplicate data.
- PUT, DELETE, and GET are idempotent methods.
- It is important for reliability and error handling.

Example: Sending DELETE /users/1 multiple times will delete the user once, not multiple times.

23 What is API testing?

- API testing is the process of testing APIs to ensure they work as expected.
- It checks functionality, reliability, performance, security, and error handling.
- Tools: Postman, Newman, RestAssured, SoapUI.
- API testing can be done earlier than UI testing.

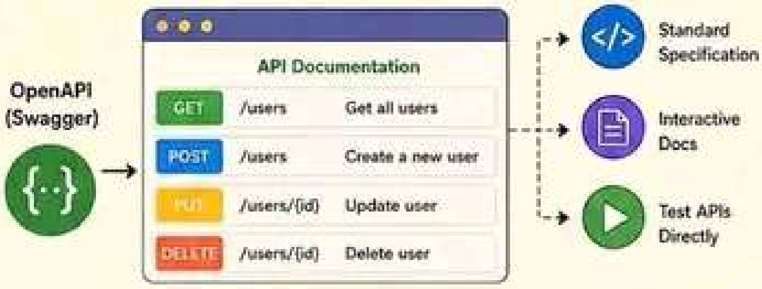
Example: Testing login API with valid and invalid credentials to verify correct responses.



24 What is Swagger / OpenAPI?

- Swagger (OpenAPI) is a specification for designing, building, documenting, and testing APIs.
- It provides a standard way to describe REST APIs.
- It generates interactive API documentation.
- Tools: Swagger UI, OpenAPI Generator.

Example: Visiting /swagger-ui.html shows all available endpoints and allows testing APIs.



25 What is API key?

- An API key is a unique code passed with the request.
- It is used to identify and authenticate the client.
- API keys help control access to the API.
- It is simple to implement but less secure than other methods if exposed.

Example: GET /data?api_key=abc123xyz
The server checks if the key is valid.

26 What is API throttling?

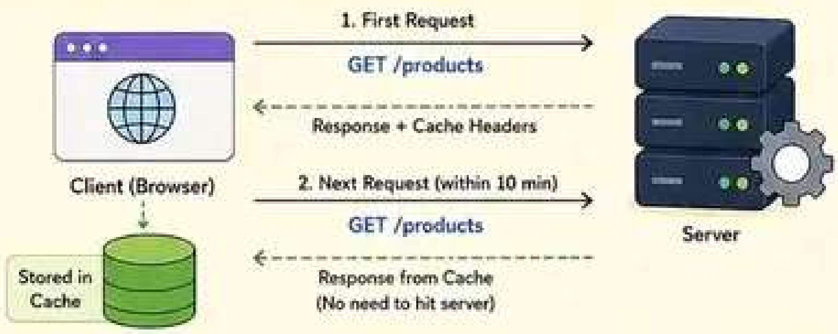
- API throttling limits the number of requests a client can make in a given time.
- It prevents abuse and ensures fair usage.
- It helps maintain API performance and stability.
- If limit is exceeded, server returns 429 Too Many Requests.

Example: Limit is 100 requests per minute per user.
101st request will get 429 Too Many Requests.

27 What is API caching?

- Caching stores responses so the same request can be served faster.
- It reduces server load and improves performance.
- Caching can be done on client-side or server-side.
- Cache is controlled using headers like Cache-Control, ETag, and Expires.

Example: Browser caches GET /products for 10 minutes.
Next time, it gets data from cache.



28 What are API best practices?

- Use meaningful and consistent endpoint names.
- Follow proper HTTP methods.
- Return appropriate status codes.
- Implement authentication and authorization.
- Validate input data.
- Use pagination for large datasets.
- Version your APIs.
- Provide clear documentation.

29 What is API rate limiting and why is it important?

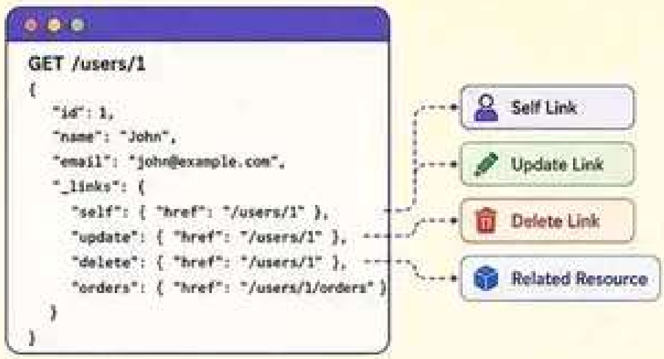
- API rate limiting restricts the number of requests a client can make within a specific time window.
- It prevents abuse, DDoS attacks, and resource exhaustion.
- It ensures fair usage for all clients.
- It helps maintain API performance and availability.

Example: 100 requests per minute per user.
The 101st request gets a 429 Too Many Requests.

30 What is HATEOAS?

- HATEOAS stands for Hypermedia As The Engine Of Application State.
- It is a constraint of REST API.
- The server provides links in responses to guide clients on what actions they can take next.
- It makes the API more discoverable and flexible.

Example: A user resource response may include links to update, delete, or view orders.



31 What are common HTTP status codes?

2xx Success
- 200 OK
- 201 Created
- 204 No Content

3xx Redirection
- 301 Moved Permanently
- 302 Found
- 304 Not Modified

4xx Client Error
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 429 Too Many Requests

5xx Server Error
- 500 Internal Server Error
- 502 Bad Gateway
- 503 Service Unavailable
- 504 Gateway Timeout

32 What is the difference between authentication and authorization?

Authentication (Who are you?)
- Verifies the identity of the user.
- Answers: "Who are you?"
- Example: Login with username and password, JWT token validation.
- Ensures the user is who they claim to be.

Authorization (What can you do?)
- Determines what the authenticated user is allowed to do.
- Answers: "What are you allowed to do?"
- Example: Accessing /admin panel, deleting a user.
- Ensures the user has permission to access a resource.

33 What is API versioning and why is it used?

- API versioning is the practice of managing changes to an API without breaking existing integrations.
- It allows you to introduce new features, fix issues, and improve the API.
- Common ways: URL path, query parameter, custom header.
- It ensures backward compatibility.

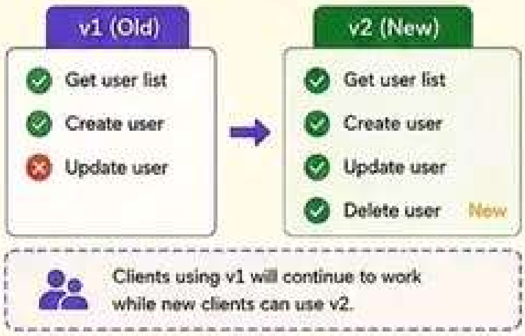
Example:

URL Path Versioning

```
v1:  /api/v1/users
v2:  /api/v2/users
```

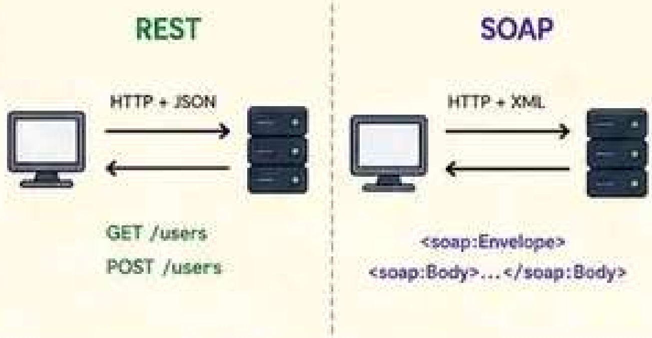
Query Parameter Versioning

```
/api/users?version=1
/api/users?version=2
```



34 What is the difference between REST and SOAP?

Feature	REST	SOAP
Full Form	Representational State Transfer	Simple Object Access Protocol
Protocol	HTTP	HTTP, SMTP, etc.
Data Format	JSON, XML, Text	XML only
Performance	Faster, lightweight	Slower, heavyweight
Standards	Uses HTTP methods (GET, POST, PUT, DELETE)	Uses its own standards
Use Case	Web, Mobile, Microservices	Enterprise, Banking, Legacy systems
Ease of Use	Simple and easy to implement	Complex and strict



35 What is a web API?

- A web API is an API that is accessed over the web using HTTP.
- It allows different software applications to communicate.
- It is platform-independent and language-independent.
- It uses standard HTTP methods and status codes.
- Examples: Twitter API, Google Maps API, GitHub API.

Example:

HTTP Methods in Web APIs	
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Delete data

36 What are API best practices?

- Use meaningful and consistent naming.
- Follow standard HTTP methods and status codes.
- Implement authentication and authorization.
- Validate input and handle errors properly.
- Use pagination, filtering, and sorting for large data.
- Version your API.
- Provide clear documentation.
- Monitor and log API usage.

1. Consistent Naming
Use clear, plural nouns for resource names.
`/api/v1/users`

2. Authentication
Use API keys, OAuth2, or JWT to secure APIs.

3. Input Validation
Validate all inputs and return proper error messages.

4. Error Handling
Return meaningful errors with proper status codes and messages.
Example: 404 Not Found

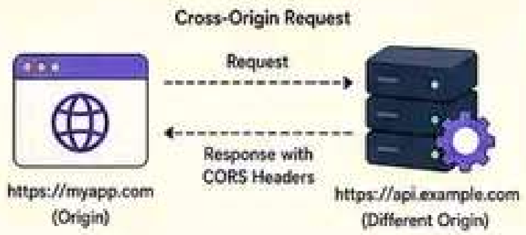
5. Pagination
Use limit & offset or page number for large responses.
Example: `?page=1&limit=10`

6. Documentation
Provide interactive and up-to-date API documentation.

37 What is CORS in API?

- CORS (Cross-Origin Resource Sharing) is a browser security feature.
- It allows or restricts resources on a web page to be requested from another domain.
- It is controlled using HTTP headers.
- It helps APIs securely allow cross-origin requests.

Example: Allowing https://myapp.com to access your API at https://api.example.com



Important CORS Headers

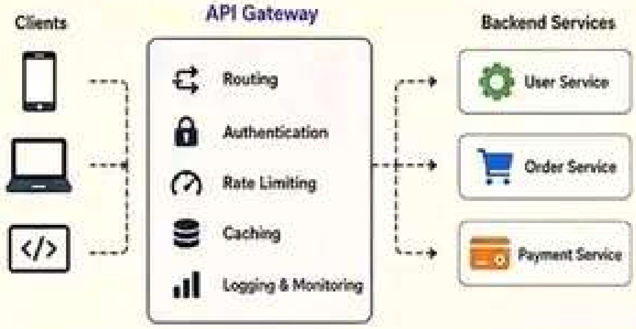
- Access-Control-Allow-Origin: https://myapp.com
- Access-Control-Allow-Methods: GET, POST, PUT, DELETE
- Access-Control-Allow-Headers: Content-Type, Authorization
- Access-Control-Allow-Credentials: true
- Access-Control-Max-Age: 3600

Without proper CORS configuration, browsers will block the response and the API cannot be accessed from another origin.

38 What is API gateway?

- An API Gateway is a server that acts as an entry point for all client requests.
- It handles routing, authentication, rate limiting, caching, logging, and more.
- It sits between clients and multiple backend services.
- It improves security, scalability, and maintainability.

Example: AWS API Gateway, Kong, Apigee, Ambassador, Tyk.



Benefits

- Single entry point
- Security & Access control
- Request/Response transformation
- Rate limiting & Throttling
- Monitoring & Analytics
- Scalability & Reliability

39 What is throttling vs rate limiting?

Throttling

- Limits the rate of requests for a client or user.
- Allows a certain number of requests in a given time window.
- Extra requests are rejected or delayed.
- Used to prevent abuse and ensure stability.

Example: 100 requests per minute per user.

VS

Rate Limiting

- Limits the overall rate of requests to a system.
- Applies to all clients collectively.
- Protects the system from being overwhelmed.
- Implemented at server or API Gateway level.

Example: 1000 requests per second for the API.

Comparison

Aspect	Throttling	Rate Limiting
Scope	Per user / client	System-wide
Purpose	Prevent user abuse	Protect system
Applied At	User / Client level	Server / Gateway level
Example	100 req/min per user	1000 req/sec globally

40 What are API security best practices?

- Use HTTPS to encrypt data in transit.
- Implement authentication (OAuth2, JWT, API Keys).
- Use authorization to control access.
- Validate and sanitize all inputs.
- Use rate limiting and throttling.
- Log and monitor API requests.
- Keep dependencies and libraries updated.
- Follow the principle of least privilege.

Pro Tip

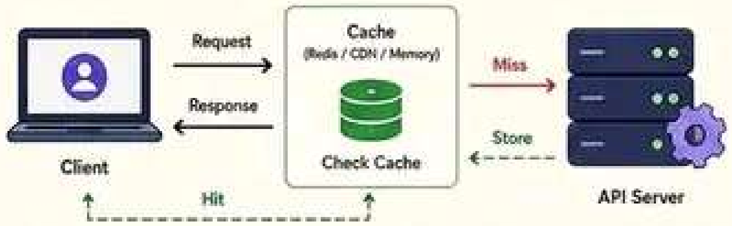
Security is an ongoing process. Regularly audit your APIs and stay updated with the latest threats and best practices.

41 What is API caching?

- API caching stores responses temporarily.
- It reduces response time and server load.
- It improves performance and scalability.
- Cached data is returned for identical requests within a specific time (TTL).
- Cache invalidation ensures fresh data.

Example:
You request product details. The API caches the response for 60 seconds. If you request again within 60 seconds, you get the cached response.

How API Caching Works



Benefits | Faster Responses | Reduced Server Load | Lower Latency | Better Scalability

42 What is API idempotency?

- Idempotency means making the same request multiple times has the same effect as once.
- It prevents duplicate actions.
- Important for POST, PUT, and DELETE requests.
- Achieved using Idempotency Keys.

Example:
Placing an order with the same Idempotency-Key multiple times should create only one order.

Best Practice
Use a unique key per request and store the response for a specific time (e.g., 24 hours).

How Idempotency Works

43 What are API testing types?

1. Functional Testing
- Validates that API works as expected.
- Tests endpoints, methods, parameters, and responses.
- Example: Status codes, response body.

2. Performance Testing
- Tests API under load.
- Checks response time, throughput, and stability.
- Tools: JMeter, LoadRunner, k6.

3. Security Testing
- Checks for vulnerabilities.
- Tests authentication, authorization, input validation, and data exposure.
- Tools: OWASP ZAP, Burp Suite.

4. Reliability Testing
- Ensures API works in different conditions.
- Tests error handling, retries, and failover.
- Checks consistency and uptime.

5. Contract Testing
- Validates API contract between provider and consumer.
- Ensures requests and responses follow the agreed schema.
- Tools: Pact, Dredd.

44 What is API monitoring?

- API monitoring tracks the health and performance of APIs.
- It helps detect issues early.
- It ensures availability, reliability, and user satisfaction.
- Key metrics: latency, uptime, error rate, throughput.
- Tools: Prometheus, Grafana, Datadog, New Relic, Postman.

Example:
Monitoring can alert you if response time is high or error rate increases.

Key Metrics to Monitor

- **Latency** — Response time of the API
- **Uptime** — API availability percentage
- **Error Rate** — Percentage of failed requests
- **Throughput** — Number of requests per second
- **Active Users** — Number of active users

Set up alerts for anomalies and downtime to respond quickly.

45 API Interview Quick Summary

- **API:** Set of rules that allows applications to communicate.
- **REST:** Architectural style using HTTP methods and statelessness.
- **HTTP Methods:** GET, POST, PUT, DELETE and their purpose.
- **Status Codes:** 2xx success, 3xx redirection, 4xx client error, 5xx server error.
- **Authentication vs Authorization:** Verify identity vs grant permissions.
- **Rate Limiting vs Throttling:** Protect system vs control client usage.
- **Caching:** Improves performance by storing responses.
- **Versioning:** Manage changes without breaking existing clients.
- **Security Best Practices:** HTTPS, Auth, Input Validation, Least Privilege.
- **Testing:** Functional, Performance, Security, Reliability, Contract.
- **Monitoring:** Track metrics, logs, and set alerts.

API Lifecycle

1. **Design** — Define goals, resources, endpoints, and data models.
2. **Develop** — Build the API, implement logic and integrations.
3. **Secure** — Add authentication, authorization, and validation.
4. **Deploy** — Release the API to a server or API Gateway.
5. **Monitor** — Monitor performance, errors, and usage.
6. **Evolve** — Version, improve, and add new features.

Key Takeaway

Good APIs are easy to use, secure, reliable, and performant. They are the building blocks of modern software systems.

> *"Designing great APIs is not just about technology, it's about solving problems for developers and users."*

46 API Tools You Should Know

- **Postman** — Test APIs, create collections, automate tests, and mock servers.
- **Swagger / OpenAPI** — Design, document, and test APIs using OpenAPI Specification.
- **Insomnia** — Simple and powerful API client for testing and debugging.
- **JMeter** — Performance testing tool for APIs and web services.
- **OWASP ZAP** — Security testing tool to find vulnerabilities in APIs.
- **Datadog** — Monitoring, logging, and analytics for APIs and applications.
- **Grafana** — Data visualization and monitoring dashboards.

47 Common API Interview Questions

Basics
- What is an API?
- What is REST?
- What are HTTP methods?
- What are HTTP status codes?
- What is a URI vs URL?
- What is JSON?

Concepts
- What is the difference between REST and SOAP?
- What is statelessness?
- What is idempotency?
- What is pagination?
- What is API versioning and why?

Security
- What is authentication?
- What is authorization?
- What is OAuth 2.0?
- What is JWT?
- What is CORS?
- What are API security best practices?

Advanced
- What is rate limiting vs throttling?
- What is caching in API?
- How do you design a scalable API?
- What are API testing types?
- How do you monitor an API?

Pro Tip
- Understand concepts clearly.
- Use real-world examples in answers.
- Explain trade-offs.
- Stay updated with API trends.
- Practice with tools.

48 API Trends to Watch

- **API Gateways** — Central point for security, routing, and management.
- **Microservices** — APIs are the backbone of microservice architectures.
- **GraphQL** — Flexible query language for APIs and data.
- **Serverless APIs** — Build and run APIs without managing servers.
- **API Management** — End-to-end lifecycle management, security, and analytics.
- **AI & APIs** — APIs powering AI/ML models and integrations.

Final Note

APIs connect systems, data, and people. Mastering APIs opens doors to countless opportunities in tech!

Keep Learning & Keep Building! 🚀

Congratulations! You've completed the API Interview Series.
Keep revising, keep building, and become an API expert! 💪